

PROGRAMACIÓN AVANZADA

Práctica de Laboratorio (PECL)

La Batalla de Hawkins: Simulación de la invasión del mundo del revés

Memoria explicativa de la práctica

Grado	Grado en Matemáticas y Computación
Curso	2025/2026 - Convocatoria Ordinaria
Alumno/a 1	Jaime Rodríguez López
Alumno/a 2	Izan Murcia Yugueros
Entorno de desarrollo	IntelliJ IDEA / Proyecto Java
Tecnología distribuida	Java RMI

Fecha de elaboración: mayo de 2026

Índice

1. Análisis de alto nivel
 2. Diseño general del sistema y sincronización
 3. Descripción de las clases principales
 4. Diagrama de clases
 5. Ejecución y funcionamiento del sistema
 6. Conclusiones
- Anexo I. Código fuente del programa

1. Análisis de alto nivel

La práctica consiste en implementar una simulación concurrente y distribuida de la ciudad de Hawkins durante la invasión del Upside Down. El sistema representa dos mundos conectados mediante portales: Hawkins, que actúa como zona segura, y el Upside Down, que contiene varias zonas peligrosas en las que se mueven los demogorgons. La simulación no tiene un final prefijado, por lo que los hilos principales permanecen activos de forma indefinida mientras se actualiza el estado en la interfaz gráfica y en el módulo remoto.

El objetivo principal es coordinar correctamente a una gran cantidad de hilos que acceden a recursos comunes. En concreto, el sistema genera 1.500 niños de forma escalonada y un demogorgon inicial, denominado Demogorgon Alpha. Posteriormente, Vecna puede generar nuevos demogorgons cada vez que se acumulan suficientes capturas en la colmena. La solución desarrollada combina programación concurrente, interfaz gráfica en Swing y acceso remoto mediante Java RMI.

1.1. Actores principales

Actor	Papel en el sistema
Niños	Son hilos que comienzan en la Calle Principal, pasan por el Sótano Byers, cruzan un portal hacia una zona del Upside Down, recogen sangre de Vecna y regresan a Hawkins para depositarla y descansar en la Radio WSQK.
Demogorgons	Son hilos enemigos que se desplazan entre las zonas inseguras del Upside Down. Cuando encuentran niños, seleccionan una víctima y realizan un ataque con una probabilidad de captura asociada a la resistencia del niño.
Vecna	No se implementa como clase independiente, sino como comportamiento del sistema: cada 8 víctimas acumuladas se genera un nuevo demogorgon.
Eleven y eventos globales	Se gestionan mediante la clase EventosAleatorios. Los eventos modifican temporalmente el comportamiento de todos los hilos: apagón, tormenta, intervención de Eleven y red mental.
Servidor RMI	Publica el estado interno de la simulación para que pueda ser consultado por un cliente remoto.
Cliente remoto	Consulta automáticamente la información del servidor RMI y permite pausar o reanudar la simulación principal.

1.2. Zonas del sistema

Zona	Descripción
Hawkins	Calle Principal, Sótano Byers y Radio WSQK. Es la parte segura del sistema y concentra el descanso, la preparación y el depósito de sangre.
Portales	Conectan Hawkins con Bosque, Laboratorio, Centro Comercial y Alcantarillado. Cada portal permite el paso de un único niño a la vez y prioriza el regreso desde el Upside Down.
Upside Down	Incluye Bosque, Laboratorio, Centro Comercial y Alcantarillado. En estas zonas coinciden niños y demogorgons.
Colmena	Almacena temporalmente los niños capturados. Durante la intervención de Eleven pueden ser liberados en función de la sangre disponible.

1.3. Flujo general de la simulación

- La clase main inicializa el logger, los eventos aleatorios, el gestor de lugares, el servidor RMI, el demogorgon inicial, el creador de niños, el creador de nuevos demogorgons y la interfaz gráfica local.
- CrearNinos instancia 1.500 objetos Nino en intervalos aleatorios entre 0,5 y 2 segundos, asignando identificadores con el formato NXXXX.
- Cada Nino repite un ciclo indefinido: Calle Principal, Sótano Byers, espera de grupo, cruce del portal, exploración del Upside Down, regreso, depósito de sangre, Radio WSQK y vuelta a la Calle Principal.
- Los Demogorgon se registran como hilos, escogen zonas del Upside Down, buscan víctimas, atacan y actualizan su contador de capturas si el ataque tiene éxito.
- EventosAleatorios activa cada cierto tiempo un evento global que modifica el comportamiento de portales, niños o demogorgons durante varios segundos.
- HawkinsServiceImpl toma una fotografía del estado de la simulación y la envía al ClienteRemoto mediante RMI.

2. Diseño general del sistema y sincronización

El diseño se organiza alrededor de una clase central, Lugares, que actúa como repositorio compartido de las zonas, portales y contadores de la simulación. Los hilos Nino y Demogorgon consultan y modifican estas estructuras durante su ejecución. Para evitar accesos inconsistentes se combinan varias herramientas de sincronización de Java, seleccionadas según el tipo de problema que resuelven: exclusión mutua, formación de grupos, comunicación entre hilos, parada remota y actualización segura de contadores.

2.1. Arquitectura concurrente

Componente	Responsabilidad
main	Arranca todos los componentes de la simulación.
Lugares	Gestiona las listas de zonas, las colas de portales, los semáforos y la liberación de niños durante la intervención de Eleven.
CrearNinos	Produce paulatinamente los 1.500 niños solicitados.
Nino	Ejecuta el ciclo vital de cada niño.
Demogorgon	Ejecuta la búsqueda, ataque y captura de niños.
CrearDemogorgons	Genera nuevos demogorgons cuando la colmena acumula 8 víctimas.
EventosAleatorios	Activa eventos globales y expone su estado mediante banderas volatile.
SimulacionGUI	Muestra el estado del sistema en tiempo real mediante Swing.
Log	Centraliza la escritura de mensajes en hawkins.txt y en la interfaz.

2.2. Arquitectura distribuida

La parte distribuida se ha implementado mediante Java RMI. El servidor se ejecuta dentro del mismo programa principal de la simulación y publica un objeto remoto de tipo `HawkinsServiceImpl`. El cliente remoto se conecta al servicio publicado en `rmi://localhost:1099/HawkinsService` y actualiza automáticamente la información mostrada mediante un `Timer` de `Swing`.

- `HawkinsService` define las operaciones remotas disponibles: obtener el estado, pausar, reanudar y consultar si la simulación está pausada.
- `HawkinsServiceImpl` implementa esa interfaz, lee las estructuras de `Lugares`, consulta el estado del evento activo y construye un objeto `EstadoHawkins` serializable.
- `EstadoHawkins` funciona como objeto de transferencia de datos. Contiene contadores de niños y demogorgons, estado de portales, sangre disponible, evento activo, estado de pausa y ranking de capturas.
- `ClienteRemoto` consulta el servicio de forma automática cada segundo y permite detener o reanudar la simulación principal sin interactuar con la ventana local.

2.3. Herramientas de sincronización utilizadas

Mecanismo	Uso en la práctica
Thread	Nino, Demogorgon, CrearNinos, CrearDemogorgons, EventosAleatorios y Lugares heredan de Thread para ejecutar comportamientos concurrentes.
Collections.synchronizedList	Se usa para proteger las listas compartidas de zonas, colmena, radio y demogorgons activos frente a accesos concurrentes.
LinkedBlockingQueue	Representa las colas de espera de los portales. Cada portal tiene una cola en sentido Hawkins -> Upside Down y otra en sentido Upside Down -> Hawkins.
Semaphore	Garantiza que por cada portal solo cruce un niño a la vez. También se usa un semáforo adicional para evitar que varios demogorgons seleccionen simultáneamente la misma víctima.
CyclicBarrier	Permite que los niños esperen a formar el tamaño de grupo exigido antes de dirigirse a cada portal: 2, 3, 4 o 2 miembros según la zona.
AtomicInteger	Gestiona de forma atómica la sangre almacenada, las víctimas acumuladas y los contadores de identificadores.
volatile	Permite que los cambios en los eventos globales sean visibles inmediatamente para los hilos que consultan sus banderas.
synchronized + wait/notifyAll	ControlPausa implementa la pausa y reanudación global. Los hilos comprueban si deben detenerse temporalmente y continúan cuando se notifica la reanudación.
ReentrantLock	Log usa un bloqueo explícito para que la escritura en el fichero hawkins.txt sea segura y ordenada aunque muchos hilos registren eventos al mismo tiempo.
RMI	Permite que el cliente remoto consulte el estado de la simulación y envíe órdenes de pausa o reanudación.

2.4. Gestión de portales

Cada portal se modela mediante dos colas: una para los niños que quieren ir desde Hawkins al Upside Down y otra para los que quieren volver desde el Upside Down a Hawkins. El hilo Lugares revisa periódicamente las colas y, si no hay apagón activo, mueve niños de los portales respetando la prioridad de vuelta a Hawkins. Además, cada portal tiene un semáforo propio que limita el cruce a un único niño durante aproximadamente un segundo.

La formación de grupos se resuelve antes de entrar en la cola del portal mediante objetos CyclicBarrier. De este modo, los niños no continúan hacia el portal hasta alcanzar el número requerido para su destino: dos para Bosque, tres para Laboratorio, cuatro para Centro Comercial y dos para Alcantarillado.

2.5. Gestión de eventos globales

Evento	Tratamiento en el programa
Tormenta del Upside Down	Los niños duplican su tiempo de permanencia en el Upside Down y los demogorgons reducen a la mitad sus tiempos de espera o ataque.
La Red Mental	Los demogorgons se dirigen a la zona con más niños en ese momento.
Intervención de Eleven	Los demogorgons quedan paralizados y se liberan niños de la colmena en función de la sangre almacenada.
Apagón del Laboratorio	Los portales dejan de mover nuevos niños mientras dura el evento; los demogorgons permanecen en la zona en la que estaban.

3. Descripción de las clases principales

A continuación se describen las clases principales implementadas en el proyecto, indicando su función dentro de la simulación y los métodos más relevantes. La explicación se centra en las clases que sostienen la concurrencia, la comunicación remota y la presentación del estado del sistema.

Clase	Responsabilidad	Métodos principales	Descripción
main	Punto de entrada del sistema.	main(String[] args).	Inicializa Log, EventosAleatorios, Lugares, ServidorRMI, Demogorgon Alpha, CrearNinos, CrearDemogorgons y SimulacionGUI.
Nino	Modelo activo de cada niño.	run(), CicloNino(), sufreAtaque(long), getNombre(), getSangre(), setSangre(int).	Gestiona el ciclo completo del niño: preparación, portal, exploración, ataque, retorno, depósito de sangre y descanso.
Demogorgon	Modelo activo de cada demogorgon.	run(), getNombre(), getCapturas().	Selecciona zonas, busca niños, ataca víctimas, incrementa capturas y se registra como demogorgon activo.
Lugares	Gestor central de zonas y portales.	run(), registrarDemogorgon(), getPortalPareja(), getSemaphoreAdecuado(), getVictimas().	Almacena listas sincronizadas, colas de portales, semáforos, barreras y contadores compartidos.
CrearNinos	Productor de niños.	run(), nextNinold().	Crea 1.500 hilos Nino con intervalos aleatorios entre 0,5 y 2 segundos.
CrearDemogorgons	Productor de demogorgons.	run(), nextDemogorgonId().	Genera un nuevo demogorgon cada vez que se acumulan 8 víctimas.
EventosAleatorios	Gestor de eventos globales.	run(), setters/getters de eventos, getDescripcionEventoActual().	Activa aleatoriamente tormenta, red mental, intervención de Eleven o apagón, manteniendo visible el evento actual para GUI y RMI.
Log	Recurso compartido de registro.	getInstance(), log(String), close(), setGuiCallback().	Escribe eventos en hawkins.txt y protege el acceso con ReentrantLock.
ControlPausa	Control global de pausa.	getInstance(), pausar(), reanudar(), esperarSiPausado(), dormir(long).	Permite detener y reanudar todos los hilos de forma coordinada, también desde el cliente remoto.
SimulacionGUI	Interfaz gráfica local.	refreshStats(), appendLog().	Muestra los contadores de Hawkins, portales, Upside Down, colmena, eventos y log en tiempo real.
HawkinsService	Interfaz remota RMI.	getEstado(), pausar(), reanudar(), estaPausado().	Define las operaciones disponibles para el cliente remoto.
HawkinsServiceImpl	Implementación del	getEstado(), pausar(),	Obtiene una instantánea del sistema,

Clase	Responsabilidad	Métodos principales	Descripción
	servicio remoto.	reanudar(), estaPausado(), obtenerRankingDemogorgons().	calcula el ranking de demogorgons y ejecuta la pausa o reanudación remota.
EstadoHawkins	Objeto de transferencia de datos.	Constructor y getters.	Agrupar todos los datos enviados desde el servidor al cliente remoto.
ServidorRMI	Publicador del servicio remoto.	iniciar(Lugares, EventosAleatorios, Log).	Crea el registro RMI en el puerto 1099 y publica HawkinsService.
ClienteRemoto	Interfaz gráfica remota.	actualizarEstado(), formatearEstado(), cambiarPausa().	Consulta automáticamente el servidor RMI y permite pausar o reanudar la ejecución principal.

3.1. Clases concurrentes

Las clases Nino y Demogorgon son las entidades activas principales. Ambas heredan de Thread y ejecutan bucles indefinidos, como exige la simulación. Nino representa el comportamiento colaborativo de los niños, mientras que Demogorgon representa el comportamiento hostil de las criaturas del Upside Down. A estas clases se suman CrearNinos y CrearDemogorgons, que actúan como productores de nuevos hilos, y EventosAleatorios, que introduce cambios globales sobre la ejecución.

3.2. Clases de coordinación

La clase Lugares centraliza el estado compartido. Esta decisión evita dispersar las estructuras de sincronización por todo el proyecto y facilita que tanto la interfaz local como el servidor RMI puedan consultar el estado actual. Además, ControlPausa y Log se han diseñado como singletons para que todos los hilos compartan el mismo control de pausa y el mismo sistema de registro.

3.3. Clases distribuidas

La parte distribuida se apoya en una interfaz remota, una implementación del servicio, un objeto serializable de estado y un cliente Swing. El servidor no modifica la lógica de la simulación, sino que expone una vista remota de sus datos y permite ejecutar dos operaciones de control: pausar y reanudar.

4. Diagrama de clases

El siguiente diagrama resume las clases principales del sistema y sus relaciones más importantes. Se muestra una visión de alto nivel para que la estructura resulte legible: se distinguen el arranque de la aplicación, la parte concurrente, los recursos compartidos, las interfaces gráficas y el módulo distribuido RMI.

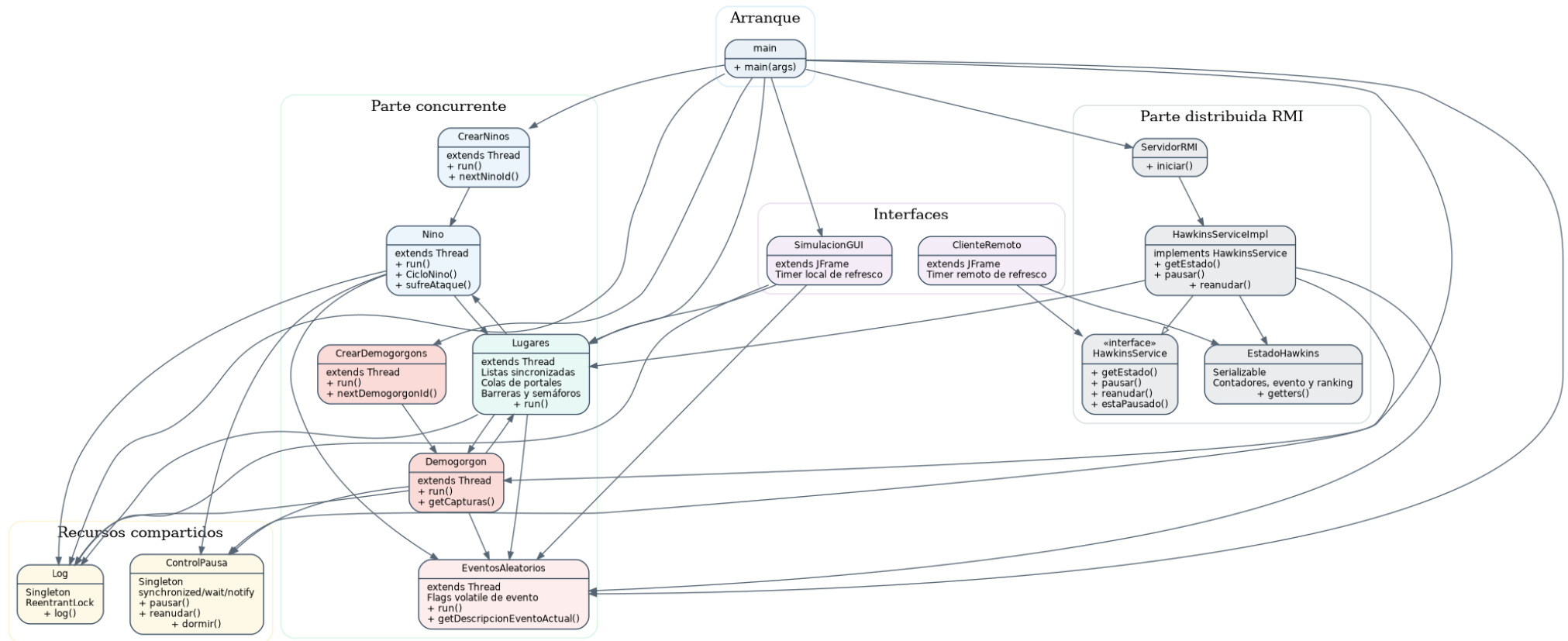


Figura 1. Diagrama de clases de alto nivel del sistema implementado.

5. Ejecución y funcionamiento del sistema

Para ejecutar la simulación principal se lanza la clase main. Durante el arranque se crea el fichero hawkins.txt, se inicializan los eventos, se ponen en marcha los portales, se publica el servicio RMI, se crea el Demogorgon Alpha y se inicia la generación de niños y nuevos demogorgons. La ventana SimulacionGUI muestra automáticamente el estado de las zonas, portales, sangre almacenada, colmena y eventos activos.

El cliente remoto se ejecuta lanzando la clase ClienteRemoto mientras el programa principal permanece activo. El cliente realiza una conexión RMI al servicio HawkinsService y actualiza cada segundo la información recibida. Además, incorpora un botón que alterna entre pausar y reanudar la simulación. Esta pausa se implementa en el servidor mediante ControlPausa, por lo que afecta a todos los hilos que consultan el estado de pausa durante sus esperas o ciclos de ejecución.

5.1. Información ofrecida por el cliente remoto

- Número total de niños en Hawkins y desglose por Calle Principal, Sótano Byers y Radio WSQK.
- Número de niños esperando en cada portal, diferenciando el sentido Hawkins -> Upside Down y Upside Down -> Hawkins.
- Número de niños y demogorgons en Bosque, Laboratorio, Centro Comercial y Alcantarillado.
- Número de niños capturados en la colmena.
- Sangre de Vecna disponible.
- Ranking de los tres demogorgons con más capturas.
- Evento global activo y tiempo restante aproximado.
- Estado de pausa de la simulación principal.

5.2. Registro de actividad

El sistema registra los acontecimientos relevantes en el fichero hawkins.txt. Cada línea incluye una marca temporal con fecha, hora, minuto y segundo, seguida del mensaje descriptivo. Al centralizar el registro en la clase Log y protegerlo con ReentrantLock, se evita que varios hilos escriban al mismo tiempo de forma desordenada.

6. Conclusiones

La solución implementada cumple el objetivo principal de la práctica: simular de forma concurrente la batalla de Hawkins y añadir un módulo distribuido que permite consultar y controlar el estado del sistema en tiempo real. El diseño separa las responsabilidades entre entidades activas, gestor de zonas, eventos globales, registro, interfaz local y servicio remoto.

El uso combinado de listas sincronizadas, colas bloqueantes, semáforos, barreras cíclicas, contadores atómicos, banderas volátiles y control de pausa permite resolver los principales problemas de coordinación que plantea el enunciado. Además, la integración con Java RMI proporciona una forma clara de consultar el estado desde otro programa sin detener la simulación local.

En conjunto, el proyecto representa una práctica completa de programación avanzada, ya que combina concurrencia, sincronización, interfaz gráfica, comunicación distribuida y gestión de recursos compartidos en un mismo sistema.

Anexo I. Código fuente del programa

En este anexo se incluye el código fuente de las clases Java desarrolladas para la práctica. El código forma parte de la memoria como exige el enunciado de entrega.

ClienteRemoto.java

```
import javax.swing.*;
import javax.swing.border.EmptyBorder;
import java.awt.*;
import java.rmi.Naming;

public class ClienteRemoto extends JFrame {
    private HawkinsService servicio;
    private final JLabel lblConexion = new JLabel("Conectando...");
    private final JLabel lblPausa = new JLabel("-");
    private final JTextArea areaEstado = new JTextArea();
    private final JButton btnPausarReanudar = new JButton("Pausar");
    private Timer timer;

    public ClienteRemoto() {
        super("Cliente remoto - La Batalla de Hawkins");
        configurarVentana();
        conectar();
        iniciarActualizacionAutomatica();
    }

    private void configurarVentana() {
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(720, 650);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout(8, 8));

        JPanel cabecera = new JPanel(new BorderLayout(8, 8));
        cabecera.setBorder(new EmptyBorder(10, 10, 0, 10));
        lblConexion.setFont(new Font("SansSerif", Font.BOLD, 13));
        lblPausa.setHorizontalAlignment(SwingConstants.RIGHT);
        cabecera.add(lblConexion, BorderLayout.WEST);
        cabecera.add(lblPausa, BorderLayout.EAST);

        areaEstado.setEditable(false);
        areaEstado.setFont(new Font("Monospaced", Font.PLAIN, 13));
        areaEstado.setMargin(new Insets(10, 10, 10, 10));

        JPanel pie = new JPanel(new FlowLayout(FlowLayout.RIGHT));
        pie.setBorder(new EmptyBorder(0, 10, 10, 10));
        btnPausarReanudar.addActionListener(e -> cambiarPausa());
        pie.add(btnPausarReanudar);

        add(cabecera, BorderLayout.NORTH);
        add(new JScrollPane(areaEstado), BorderLayout.CENTER);
        add(pie, BorderLayout.SOUTH);
    }

    private void conectar() {
        try {
            servicio = (HawkinsService) Naming.lookup("rmi://localhost:1099/HawkinsService");
            lblConexion.setText("Conectado al servidor RMI de Hawkins");
        } catch (Exception e) {
            lblConexion.setText("No se pudo conectar al servidor RMI");
            areaEstado.setText("Comprueba que el programa principal esté ejecutándose.\n\n" + e.getMessage());
            btnPausarReanudar.setEnabled(false);
        }
    }

    private void iniciarActualizacionAutomatica() {
        timer = new Timer(1000, e -> actualizarEstado());
        timer.start();
    }

    private void actualizarEstado() {
        if (servicio == null) {
            return;
        }

        try {
            EstadoHawkins estado = servicio.getEstado();
            lblPausa.setText(estado.isPausado() ? "Estado: PAUSADO" : "Estado: EN EJECUCION");
            btnPausarReanudar.setText(estado.isPausado() ? "Reanudar" : "Pausar");
            areaEstado.setText(formatearEstado(estado));
        } catch (Exception e) {
            lblConexion.setText("Conexion perdida con el servidor RMI");
            areaEstado.setText("Error consultando el estado remoto:\n" + e.getMessage());
        }
    }

    private String formatearEstado(EstadoHawkins e) {
        StringBuilder sb = new StringBuilder();
        sb.append("ULTIMA ACTUALIZACION: ").append(e.getFechaActualizacion()).append("\n\n");

        sb.append("HAWKINS\n");
        sb.append("  Total en Hawkins: ").append(e.getNinosHawkins()).append("\n");
        sb.append("  Calle Principal: ").append(e.getNinosCallePrincipal()).append("\n");
        sb.append("  Sotano Byers: ").append(e.getNinosSotanoByers()).append("\n");
        sb.append("  Radio WSQK: ").append(e.getNinosRadioWSQK()).append("\n\n");

        sb.append("PORTALES (Normal -> Upside Down | Upside Down -> Hawkins)\n");
        sb.append(String.format("  Bosque: %4d | %4d\n", e.getPortalBosqueNormal(), e.getPortalBosqueUpsideDown()));
        sb.append(String.format("  Laboratorio: %4d | %4d\n", e.getPortalLaboratorioNormal(), e.getPortalLaboratorioUpsideDown()));
        sb.append(String.format("  Centro Comercial: %4d | %4d\n", e.getPortalCentroNormal(), e.getPortalCentroUpsideDown()));
    }
}
```

```

sb.append(String.format(" Alcantarillado:      %4d | %4d%n%n", e.getPortalAlcantarilladoNormal(), e.getPortalAlcantarilladoUpsideDown()));

sb.append("UPSIDE DOWN\n");
sb.append(String.format(" Bosque:          ninos=%4d  demogorgons=%4d%n", e.getNinosBosque(), e.getDemogorgonsBosque()));
sb.append(String.format(" Laboratorio:      ninos=%4d  demogorgons=%4d%n", e.getNinosLaboratorio(), e.getDemogorgonsLaboratorio()));
sb.append(String.format(" Centro Comercial:   ninos=%4d  demogorgons=%4d%n", e.getNinosCentroComercial(),
e.getDemogorgonsCentroComercial()));
sb.append(String.format(" Alcantarillado:      ninos=%4d  demogorgons=%4d%n", e.getNinosAlcantarillado(),
e.getDemogorgonsAlcantarillado()));
sb.append(" Colmena:          ").append(e.getNinosColmena()).append(" capturados\n");
sb.append(" Demogorgons activos: ").append(e.getDemogorgonsActivos()).append("\n\n");

sb.append("RECURSOS Y EVENTOS\n");
sb.append(" Sangre disponible:  ").append(e.getSangreDisponible()).append("\n");
sb.append(" Evento activo:      ").append(e.getEventoActivo()).append("\n\n");

sb.append("RANKING TOP 3 DEMOGORGONS\n");
String[] ranking = e.getRankingDemogorgons();
if (ranking.length == 0) {
    sb.append(" Sin demogorgons registrados todavia\n");
} else {
    for (int i = 0; i < ranking.length; i++) {
        sb.append(" ").append(i + 1).append(" ").append(ranking[i]).append("\n");
    }
}

return sb.toString();
}

private void cambiarPausa() {
    try {
        if (servicio.estaPausado()) {
            servicio.reanudar();
        } else {
            servicio.pausar();
        }
    } catch (Exception e) {
        JOptionPane.showMessageDialog(this,
            "No se pudo cambiar el estado de pausa:\n" + e.getMessage(),
            "Error RMI",
            JOptionPane.ERROR_MESSAGE);
    }
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new ClienteRemoto().setVisible(true));
}
}

```

ControlPausa.java

```

public class ControlPausa {
    private static final ControlPausa INSTANCE = new ControlPausa();
    private boolean pausado = false;

    private ControlPausa() {
    }

    public static ControlPausa getInstance() {
        return INSTANCE;
    }

    public synchronized void pausar() {
        pausado = true;
    }

    public synchronized void reanudar() {
        pausado = false;
        notifyAll();
    }

    public synchronized boolean estaPausado() {
        return pausado;
    }

    public synchronized void esperarSiPausado() throws InterruptedException {
        while (pausado) {
            wait();
        }
    }

    public void dormir(long milisegundos) throws InterruptedException {
        long fin = System.currentTimeMillis() + milisegundos;
        while (true) {
            esperarSiPausado();
            long restante = fin - System.currentTimeMillis();
            if (restante <= 0) {
                return;
            }
            Thread.sleep(Math.min(100, restante));
        }
    }
}

```

CrearDemogorgons.java

```

import java.util.concurrent.Semaphore;
import java.util.concurrent.atomic.AtomicInteger;

```

```

public class CrearDemogorgons extends Thread {
    private Lugares lugares;
    public AtomicInteger contadorDemogorgons = new AtomicInteger(1);
    public EventosAleatorios eventos;
    public Log log;
    Semaphore semaforo = new Semaphore(1);

    public CrearDemogorgons(Lugares lugares,EventosAleatorios eventos, Log log, Semaphore semaforo) {
        this.lugares = lugares;
        this.eventos = eventos;
        this.log = log;
        this.semaforo = semaforo;
    }

    @Override
    public void run() {
        while (true) {
            try {
                ControlPausa.getInstance().esperarSiPausado();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
            if(lugares.acumuladorVictimas.get()>=8){
                String demogorgonId = nextDemogorgonId();
                Demogorgon demogorgon = new Demogorgon(demogorgonId,this.lugares, this.eventos, this.log, this.semaforo);
                lugares.acumuladorVictimas.addAndGet(-8);
                log.log("El demogorgon "+demogorgonId+" se une a la batalla" );
                demogorgon.start();
            }
            try {
                ControlPausa.getInstance().dormir(200);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
    }

    public String nextDemogorgonId() {
        return String.format("D%04d", contadorDemogorgons.getAndIncrement());
        //Devuelve un string con el formato deseado
    }
}

```

CrearNinos.java

```

import java.util.concurrent.atomic.AtomicInteger;

public class CrearNinos extends Thread{
    public int NinosTotales=1500;
    public AtomicInteger contadorNinos=new AtomicInteger(0);
    public Lugares lugares;
    public EventosAleatorios eventos;
    public Log log;
    public CrearNinos(Lugares lugares,EventosAleatorios eventos,Log log) {
        this.lugares = lugares;
        this.eventos = eventos;
        this.log = log;
    }

    @Override
    public void run(){
        for (int i = 0; i < NinosTotales; i++) {
            //Logear que empiezan a crearse niños
            try {
                ControlPausa.getInstance().esperarSiPausado();
                String idNino = nextNinoId();
                Nino nino = new Nino(idNino,lugares,this.eventos,this.log);
                nino.start();
                log.log("El niño "+idNino+" se une a la batalla");
                int CooldownCreacionNinos = 500 + (int)(Math.random() * (2000 - 500 + 1)); //Creacion del Cooldown
                ControlPausa.getInstance().dormir(CooldownCreacionNinos);
                //Logear que se ha creado un niño y su id.
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
        }
        //Logear que todos los niños han sido creados
    }

    public String nextNinoId() {
        return String.format("N%04d", contadorNinos.getAndIncrement());
        //Devuelve un string con el formato deseado
    }
}

```

Demogorgon.java

```

import java.util.List;
import java.util.concurrent.Semaphore;

```

```

public class Demogorgon extends Thread {

    public String Nombre; //DXXXX
    public Lugares lugares;
    public EventosAleatorios eventos;
    public Log log;
    private int capturas=0;
    private Semaphore exclusionEscogerNinos;

    public Demogorgon(Lugares lugares,EventosAleatorios eventos,Log log,Semaphore exclusionEscogerNinos) {
        this.Nombre= "D0000";
        this.lugares=lugares;
        this.eventos = eventos;
        this.log = log;
        this.exclusionEscogerNinos = exclusionEscogerNinos;
        this.lugares.registrarDemogorgon(this);
    }
    public Demogorgon(String Nombre, Lugares lugares, EventosAleatorios eventos, Log log, Semaphore exclusionEscogerNinos) {
        this.Nombre = Nombre;
        this.lugares=lugares;
        this.eventos= eventos;
        this.log = log;
        this.exclusionEscogerNinos = exclusionEscogerNinos;
        this.lugares.registrarDemogorgon(this);
    }

    public String getNombre() {
        return Nombre;
    }

    public int getCapturas() {
        return capturas;
    }

    @Override
    public void run() {
        List<Demogorgon> destino=null;
        while (true) {
            try {
                ControlPausa.getInstance().esperarSiPausado();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
            while(eventos.getIntervencionDeEleven()){
                try {
                    ControlPausa.getInstance().dormir(1);
                } catch (InterruptedException e) {
                    throw new RuntimeException(e);
                }
            }
            try {
                if(!eventos.getLaRedMental()){
                    if (!eventos.getApagonLaboratorio()){
                        int lugar = (int) (Math.random() * {4});

                        switch (lugar) {
                            case 0:
                                destino = lugares.getBosqueUpsideDownDemogorgon();
                                log.log("El demogorgon "+ Nombre+" se dirige al Bosque");
                                break;
                            case 1:
                                destino = lugares.getAlcantarilladoUpsideDownDemogorgon();
                                log.log("El demogorgon "+ Nombre+" se dirige al Alcantarillado");
                                break;
                            case 2:
                                destino = lugares.getCentroComercialUpsideDownDemogorgon();
                                log.log("El demogorgon "+ Nombre+" se dirige al Centro Comercial");
                                break;
                            case 3:
                                destino = lugares.getLaboratorioUpsideDownDemogorgon();
                                log.log("El demogorgon "+ Nombre+" se dirige al Laboratorio");
                                break;
                        }
                    }else{
                        if(destino == null){
                            destino = lugares.getLaboratorioUpsideDownDemogorgon();//si justo se inicializa el evento al crearse el demogorgon no tendra
                            desino, entonces le asigno uno aleatorio
                        }
                        //sigue en el mismo destino
                    }
                }
            }
            else{
                List<Nino> max=null;
                String mensaje=null;
                if (lugares.getLaboratorioUpsideDown().size()>lugares.getAlcantarilladoUpsideDown().size()) {
                    max = lugares.getLaboratorioUpsideDown();
                    destino = lugares.getLaboratorioUpsideDownDemogorgon();
                    mensaje="El demogorgon "+ Nombre+" se dirige al Laboratorio";
                }
                else{
                    max = lugares.getAlcantarilladoUpsideDown();
                    destino = lugares.getAlcantarilladoUpsideDownDemogorgon();
                    mensaje = "El demogorgon "+ Nombre+" se dirige al Alcantarillado";
                }
                if(max.size()< lugares.getBosqueUpsideDown().size()){
                    max = lugares.getBosqueUpsideDown();
                    destino = lugares.getBosqueUpsideDownDemogorgon();
                    mensaje = "El demogorgon "+ Nombre+" se dirige al Bosque";
                }
            }
        }
    }
}

```

```

    }
    if(max.size() < lugares.getCentroComercialUpsideDown().size()){
        destino = lugares.getCentroComercialUpsideDownDemogorgon();
        mensaje = "El demogorgon " + Nombre + " se dirige al Centro Comercial";
    }
    log.log(mensaje);
}
destino.add(this);
List<Nino> victimas = lugares.getVictimas(destino);
exclusionEscogerNinos.acquire();
if(victimas.isEmpty()){
    exclusionEscogerNinos.release();
    int TiempoEnZona = 4000 + (int) (Math.random() * (5000 - 4000 + 1));
    if(eventos.getTormentaUpsideDown()){
        TiempoEnZona = TiempoEnZona / 2;
    }
    ControlPausa.getInstance().dormir(TiempoEnZona);
}else{
    int numNinosEnZona = victimas.size();
    int ninoVictimaEnArray = (int) (Math.random() * numNinosEnZona);
    Nino ninoVictima = victimas.get(ninoVictimaEnArray);
    victimas.remove(ninoVictimaEnArray);

    int longitudAtaque = 500 + (int) (Math.random() * (1500 - 500 + 1));
    if (eventos.getTormentaUpsideDown()) {
        longitudAtaque = longitudAtaque / 2;
    }
    log.log("El demogorgon " + Nombre + " ataca a niño " + ninoVictima.getNombre() + ".");
    exclusionEscogerNinos.release();
    if(!ninoVictima.sufreAtaque(longitudAtaque)){

        int longitudPuestaANinoEnColmena = 500 + (int) (Math.random() * (1000 - 500 + 1));
        ControlPausa.getInstance().dormir(longitudPuestaANinoEnColmena);
        lugares.getLaColmena().add(ninoVictima);
        lugares.acumuladorVictimas.incrementAndGet();
        capturas++;
        log.log("El ataque de " + Nombre + " a " + ninoVictima.getNombre() + " ha sido realizado correctamente.(Capturas: "+capturas+"");

    }else {
        victimas.add(ninoVictima);
        log.log("El ataque de " + Nombre + " a " + ninoVictima.getNombre() + " ha fallado.(Capturas: "+capturas+"");
    }
}
} catch (InterruptedException e) {
    throw new RuntimeException(e);
}
destino.remove(this);
}
}
}
}
}

```

EstadoHawkins.java

```

import java.io.Serializable;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class EstadoHawkins implements Serializable {
    private static final long serialVersionUID = 1L;

    private final int ninosHawkins;
    private final int ninosCallePrincipal;
    private final int ninosSotanoByers;
    private final int ninosRadioWSQK;

    private final int portalBosqueNormal;
    private final int portalBosqueUpsideDown;
    private final int portalLaboratorioNormal;
    private final int portalLaboratorioUpsideDown;
    private final int portalCentroNormal;
    private final int portalCentroUpsideDown;
    private final int portalAlcantarilladoNormal;
    private final int portalAlcantarilladoUpsideDown;

    private final int ninosBosque;
    private final int ninosLaboratorio;
    private final int ninosCentroComercial;
    private final int ninosAlcantarillado;
    private final int ninosColmena;

    private final int demogorgonsBosque;
    private final int demogorgonsLaboratorio;
    private final int demogorgonsCentroComercial;
    private final int demogorgonsAlcantarillado;
    private final int demogorgonsActivos;

    private final int sangreDisponible;
    private final String eventoActivo;
    private final boolean pausado;
    private final String[] rankingDemogorgons;
    private final String fechaActualizacion;

    public EstadoHawkins(
        int ninosHawkins,
        int ninosCallePrincipal,
        int ninosSotanoByers,

```

```

        int ninosRadioWSQK,
        int portalBosqueNormal,
        int portalBosqueUpsideDown,
        int portalLaboratorioNormal,
        int portalLaboratorioUpsideDown,
        int portalCentroNormal,
        int portalCentroUpsideDown,
        int portalAlcantarilladoNormal,
        int portalAlcantarilladoUpsideDown,
        int ninosBosque,
        int ninosLaboratorio,
        int ninosCentroComercial,
        int ninosAlcantarillado,
        int ninosColmena,
        int demogorgonsBosque,
        int demogorgonsLaboratorio,
        int demogorgonsCentroComercial,
        int demogorgonsAlcantarillado,
        int demogorgonsActivos,
        int sangreDisponible,
        String eventoActivo,
        boolean pausado,
        String[] rankingDemogorgons) {
    this.ninosHawkins = ninosHawkins;
    this.ninosCallePrincipal = ninosCallePrincipal;
    this.ninosSotanoByers = ninosSotanoByers;
    this.ninosRadioWSQK = ninosRadioWSQK;
    this.portalBosqueNormal = portalBosqueNormal;
    this.portalBosqueUpsideDown = portalBosqueUpsideDown;
    this.portalLaboratorioNormal = portalLaboratorioNormal;
    this.portalLaboratorioUpsideDown = portalLaboratorioUpsideDown;
    this.portalCentroNormal = portalCentroNormal;
    this.portalCentroUpsideDown = portalCentroUpsideDown;
    this.portalAlcantarilladoNormal = portalAlcantarilladoNormal;
    this.portalAlcantarilladoUpsideDown = portalAlcantarilladoUpsideDown;
    this.ninosBosque = ninosBosque;
    this.ninosLaboratorio = ninosLaboratorio;
    this.ninosCentroComercial = ninosCentroComercial;
    this.ninosAlcantarillado = ninosAlcantarillado;
    this.ninosColmena = ninosColmena;
    this.demogorgonsBosque = demogorgonsBosque;
    this.demogorgonsLaboratorio = demogorgonsLaboratorio;
    this.demogorgonsCentroComercial = demogorgonsCentroComercial;
    this.demogorgonsAlcantarillado = demogorgonsAlcantarillado;
    this.demogorgonsActivos = demogorgonsActivos;
    this.sangreDisponible = sangreDisponible;
    this.eventoActivo = eventoActivo;
    this.pausado = pausado;
    this.rankingDemogorgons = rankingDemogorgons;
    this.fechaActualizacion = LocalDateTime.now().format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss"));
}

public int getNinosHawkins() { return ninosHawkins; }
public int getNinosCallePrincipal() { return ninosCallePrincipal; }
public int getNinosSotanoByers() { return ninosSotanoByers; }
public int getNinosRadioWSQK() { return ninosRadioWSQK; }
public int getPortalBosqueNormal() { return portalBosqueNormal; }
public int getPortalBosqueUpsideDown() { return portalBosqueUpsideDown; }
public int getPortalLaboratorioNormal() { return portalLaboratorioNormal; }
public int getPortalLaboratorioUpsideDown() { return portalLaboratorioUpsideDown; }
public int getPortalCentroNormal() { return portalCentroNormal; }
public int getPortalCentroUpsideDown() { return portalCentroUpsideDown; }
public int getPortalAlcantarilladoNormal() { return portalAlcantarilladoNormal; }
public int getPortalAlcantarilladoUpsideDown() { return portalAlcantarilladoUpsideDown; }
public int getNinosBosque() { return ninosBosque; }
public int getNinosLaboratorio() { return ninosLaboratorio; }
public int getNinosCentroComercial() { return ninosCentroComercial; }
public int getNinosAlcantarillado() { return ninosAlcantarillado; }
public int getNinosColmena() { return ninosColmena; }
public int getDemogorgonsBosque() { return demogorgonsBosque; }
public int getDemogorgonsLaboratorio() { return demogorgonsLaboratorio; }
public int getDemogorgonsCentroComercial() { return demogorgonsCentroComercial; }
public int getDemogorgonsAlcantarillado() { return demogorgonsAlcantarillado; }
public int getDemogorgonsActivos() { return demogorgonsActivos; }
public int getSangreDisponible() { return sangreDisponible; }
public String getEventoActivo() { return eventoActivo; }
public boolean isPausado() { return pausado; }
public String[] getRankingDemogorgons() { return rankingDemogorgons; }
public String getFechaActualizacion() { return fechaActualizacion; }
}

```

EventosAleatorios.java

```

public class EventosAleatorios extends Thread {
    /*
    • INTERVENCIÓN DE ELEVEN:
    o Eleven usa sus poderes para liberar tantos niños capturados en la COLMENA como
    unidades de sangre de Vecna hayan sido recolectadas en ese tiempo, y regresan a
    la CALLE PRINCIPAL de Hawkins.
    o Durante este evento, todos los demogorgons quedan paralizados (no atacan ni se
    mueven) durante la duración del evento.
    *****/

    private volatile boolean TormentaUpsideDown=false;
    private volatile boolean LaRedMental=false;
    private volatile boolean IntervencionDeEleven=false;
    private volatile boolean ApagonLaboratorio=false;
    private Log log;
    private volatile String eventoActivo = "Sin evento activo";
    private volatile long finEventoMillis = 0;

```

```

public EventosAleatorios(Log log) {
    this.log = log;
}

public boolean getLaRedMental() {
    return LaRedMental;
}

public void setLaRedMental(boolean laRedMental) {
    LaRedMental = laRedMental;
}

public boolean getIntervencionDeEleven() {
    return IntervencionDeEleven;
}

public void setIntervencionDeEleven(boolean intervencionDeEleven) {
    IntervencionDeEleven = intervencionDeEleven;
}

public boolean getApagonLaboratorio() {
    return ApagonLaboratorio;
}

public void setApagonLaboratorio(boolean apagonLaboratorio) {
    ApagonLaboratorio = apagonLaboratorio;
}

public void setTormentaUpsideDown(boolean tormentaUpsideDown) {
    TormentaUpsideDown = tormentaUpsideDown;
}

public boolean getTormentaUpsideDown() {
    return TormentaUpsideDown;
}

public synchronized void iniciarEventoRemoto(String nombre, int duracionMs) {
    eventoActivo = nombre;
    finEventoMillis = System.currentTimeMillis() + duracionMs;
}

public synchronized void finalizarEventoRemoto() {
    eventoActivo = "Sin evento activo";
    finEventoMillis = 0;
}

public synchronized String getDescripcionEventoActual() {
    if (finEventoMillis <= 0 || System.currentTimeMillis() >= finEventoMillis) {
        return "Sin evento activo";
    }
    long segundos = Math.max(0, (finEventoMillis - System.currentTimeMillis()) / 1000);
    return eventoActivo + " (quedan aprox. " + segundos + " s)";
}

@Override
public void run() {
    while (true) {
        try {
            ControlPausa.getInstance().esperarSiPausado();
            int CooldownEntreEventos = 30000 + (int) (Math.random() * (60000 - 30000 + 1));
            ControlPausa.getInstance().dormir(CooldownEntreEventos);
            int Evento = (int) (Math.random() * (4));
            int DuracionEvento = 5000 + (int) (Math.random() * (10000 - 5000 + 1));
            switch (Evento) {
                case 0:
                    setTormentaUpsideDown(true);
                    iniciarEventoRemoto("TORMENTA DEL UPSIDE DOWN", DuracionEvento);
                    log.log("***EMPIEZA EL EVENTO TORMENTA DEL UPSIDE DOWN***");
                    ControlPausa.getInstance().dormir(DuracionEvento);
                    log.log("***TERMINA EL EVENTO TORMENTA DEL UPSIDE DOWN***");
                    setTormentaUpsideDown(false);
                    finalizarEventoRemoto();
                    break;
                case 1:
                    setLaRedMental(true);
                    iniciarEventoRemoto("LA RED MENTAL", DuracionEvento);
                    log.log("***EMPIEZA EL EVENTO LA RED MENTAL***");
                    ControlPausa.getInstance().dormir(DuracionEvento);
                    log.log("***TERMINA EL EVENTO LA RED MENTAL***");
                    setLaRedMental(false);
                    finalizarEventoRemoto();
                    break;
                case 2:
                    setIntervencionDeEleven(true);
                    iniciarEventoRemoto("INTERVENCION DE ELEVEN", DuracionEvento);
                    log.log("***EMPIEZA EL EVENTO INTERVENCION DE ELEVEN***");
                    ControlPausa.getInstance().dormir(DuracionEvento);
                    log.log("***TERMINA EL EVENTO INTERVENCION DE ELEVEN***");
                    setIntervencionDeEleven(false);
                    finalizarEventoRemoto();
                    break;
                case 3:
                    setApagonLaboratorio(true);
                    iniciarEventoRemoto("APAGON DEL LABORATORIO", DuracionEvento);
                    log.log("***EMPIEZA EL EVENTO EL APAGON DEL LABORATORIO***");
                    ControlPausa.getInstance().dormir(DuracionEvento);
                    log.log("***TERMINA EL EVENTO EL APAGON DEL LABORATORIO***");
                    setApagonLaboratorio(false);
                    finalizarEventoRemoto();
                    break;
            }
        } catch (InterruptedException e){
        }
    }
}

```

```

}

}

```

HawkinsService.java

```

import java.rmi.Remote;
import java.rmi.RemoteException;

public interface HawkinsService extends Remote {
    EstadoHawkins getEstado() throws RemoteException;
    void pausar() throws RemoteException;
    void reanudar() throws RemoteException;
    boolean estaPausado() throws RemoteException;
}

```

HawkinsServiceImpl.java

```

import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

public class HawkinsServiceImpl extends UnicastRemoteObject implements HawkinsService {
    private final Lugares lugares;
    private final EventosAleatorios eventos;
    private final ControlPausa controlPausa;
    private final Log log;

    public HawkinsServiceImpl(Lugares lugares, EventosAleatorios eventos, Log log) throws RemoteException {
        super();
        this.lugares = lugares;
        this.eventos = eventos;
        this.log = log;
        this.controlPausa = ControlPausa.getInstance();
    }

    @Override
    public EstadoHawkins getEstado() throws RemoteException {
        int calle = lugares.getCallePrincipal().size();
        int sotano = lugares.getSotanoByers().size();
        int radio = lugares.getRadioWSQK().size();

        int pBosqueNormal = lugares.getPortalNormalBosque().size();
        int pBosqueUD = lugares.getPortalUpsideDownBosque().size();
        int pLabNormal = lugares.getPortalNormalLaboratorio().size();
        int pLabUD = lugares.getPortalUpsideDownLaboratorio().size();
        int pCentroNormal = lugares.getPortalNormalCentroComercial().size();
        int pCentroUD = lugares.getPortalUpsideDownCentroComercial().size();
        int pAlcNormal = lugares.getPortalNormalAlcantarillado().size();
        int pAlcUD = lugares.getPortalUpsideDownAlcantarillado().size();

        int ninosHawkins = calle + sotano + radio + pBosqueNormal + pLabNormal + pCentroNormal + pAlcNormal;

        return new EstadoHawkins(
            ninosHawkins,
            calle,
            sotano,
            radio,
            pBosqueNormal,
            pBosqueUD,
            pLabNormal,
            pLabUD,
            pCentroNormal,
            pCentroUD,
            pAlcNormal,
            pAlcUD,
            lugares.getBosqueUpsideDown().size(),
            lugares.getLaboratorioUpsideDown().size(),
            lugares.getCentroComercialUpsideDown().size(),
            lugares.getAlcantarilladoUpsideDown().size(),
            lugares.getLaColmena().size(),
            lugares.getBosqueUpsideDownDemogorgon().size(),
            lugares.getLaboratorioUpsideDownDemogorgon().size(),
            lugares.getCentroComercialUpsideDownDemogorgon().size(),
            lugares.getAlcantarilladoUpsideDownDemogorgon().size(),
            lugares.getDemogorgonsActivos().size(),
            lugares.sangreAlmacenada.get(),
            eventos.getDescripcionEventoActual(),
            controlPausa.estaPausado(),
            obtenerRankingDemogorgons()
        );
    }

    @Override
    public void pausar() throws RemoteException {
        controlPausa.pausar();
        log.log("EJECUCION PAUSADA desde el cliente remoto");
    }

    @Override
    public void reanudar() throws RemoteException {
        controlPausa.reanudar();
        log.log("EJECUCION REANUDADA desde el cliente remoto");
    }

    @Override

```



```

public boolean estaPausado() throws RemoteException {
    return controlPausa.estaPausado();
}

private String[] obtenerRankingDemogorgons() {
    List<Demogorgon> copia;
    synchronized (lugares.getDemogorgonsActivos()) {
        copia = new ArrayList<>(lugares.getDemogorgonsActivos());
    }

    return copia.stream()
        .sorted(Comparator.comparingInt(Demogorgon::getCapturas).reversed())
        .limit(3)
        .map(d -> d.getNombre() + " - " + d.getCapturas() + " capturas")
        .toArray(String[]::new);
}
}

```

Log.java

```

import java.io.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.concurrent.locks.ReentrantLock;

public class Log {

    private static final Log INSTANCE = new Log();
    private final ReentrantLock lock = new ReentrantLock();
    private PrintWriter writer;
    private static final DateTimeFormatter FMT =
        DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");
    private static java.util.function.Consumer<String> guiCallback;

    private Log() {
        try {
            writer = new PrintWriter(new FileWriter("hawkins.txt", false), true);
        } catch (IOException e) {
            System.err.println("[LOGGER ERROR] No se pudo abrir hawkins.txt: " + e.getMessage());
        }
    }

    public static Log getInstance() { return INSTANCE; }

    public void log(String message) {
        String line = "[" + LocalDateTime.now().format(FMT) + "]" + message;
        lock.lock();
        try {
            if (writer != null) writer.println(line);
            System.out.println(line);
            if (guiCallback != null) guiCallback.accept(line);
        } finally {
            lock.unlock();
        }
    }

    public void close() {
        lock.lock();
        try { if (writer != null) writer.close(); }
        finally { lock.unlock(); }
    }

    public static void setGuiCallback(java.util.function.Consumer<String> cb) { guiCallback = cb; }
}

```

Lugares.java

```

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class Lugares extends Thread{
    public EventosAleatorios eventos;
    public Log log;
    public Lugares(EventosAleatorios eventos, Log log) {
        this.eventos=eventos;
        this.log=log;
    }

    public AtomicInteger sangreAlmacenada=new AtomicInteger(0);
    public AtomicInteger acumuladorVictimas=new AtomicInteger(0);

    private final List<Nino> callePrincipal = Collections.synchronizedList(new ArrayList<>());
    private final List<Nino> sotanoByers = Collections.synchronizedList(new ArrayList<>());

    private final List<Nino> BosqueUpsideDown = Collections.synchronizedList(new ArrayList<>());
    private final List<Nino> LaboratorioUpsideDown = Collections.synchronizedList(new ArrayList<>());
    private final List<Nino> AlcantarilladoUpsideDown = Collections.synchronizedList(new ArrayList<>());
    private final List<Nino> CentroComercialUpsideDown = Collections.synchronizedList(new ArrayList<>());

    private final List<Demogorgon> BosqueUpsideDownDemogorgon = Collections.synchronizedList(new ArrayList<>());
    private final List<Demogorgon> LaboratorioUpsideDownDemogorgon = Collections.synchronizedList(new ArrayList<>());
    private final List<Demogorgon> AlcantarilladoUpsideDownDemogorgon = Collections.synchronizedList(new ArrayList<>());
    private final List<Demogorgon> CentroComercialUpsideDownDemogorgon = Collections.synchronizedList(new ArrayList<>());

    private final List<Nino> LaColmena = Collections.synchronizedList(new ArrayList<>());
    private final List<Demogorgon> demogorgonsActivos = Collections.synchronizedList(new ArrayList<>());
}

```

```

private final List<Nino> RadioWSQK = Collections.synchronizedList(new ArrayList<>());
//Sincronización de niños en el sótano byers, solo continuan cuando han llegado al numero de integrantes necesarios
private final CyclicBarrier irBosque = new CyclicBarrier(2);
private final CyclicBarrier irLaboratorio= new CyclicBarrier(3);
private final CyclicBarrier irCentroComercial= new CyclicBarrier(4);
private final CyclicBarrier irAlcantarillado= new CyclicBarrier(2);

private final LinkedBlockingQueue<Nino> portalNormalBosque = new LinkedBlockingQueue<>();//Capacidad máxima
private final LinkedBlockingQueue<Nino> portalUpsideDownBosque = new LinkedBlockingQueue<>();
private final LinkedBlockingQueue<Nino> portalNormalLaboratorio = new LinkedBlockingQueue<>();
private final LinkedBlockingQueue<Nino> portalUpsideDownLaboratorio = new LinkedBlockingQueue<>();
private final LinkedBlockingQueue<Nino> portalNormalCentroComercial = new LinkedBlockingQueue<>();
private final LinkedBlockingQueue<Nino> portalUpsideDownCentroComercial = new LinkedBlockingQueue<>();
private final LinkedBlockingQueue<Nino> portalNormalAlcantarillado = new LinkedBlockingQueue<>();
private final LinkedBlockingQueue<Nino> portalUpsideDownAlcantarillado = new LinkedBlockingQueue<>();

private final Semaphore portalAlcantarillaSemaphore = new Semaphore(1);
private final Semaphore portalCentroComercialSemaphore = new Semaphore(1);
private final Semaphore portalLaboratorioSemaphore = new Semaphore(1);
private final Semaphore portalBosqueSemaphore = new Semaphore(1);

public void registrarDemogorgon(Demogorgon demogorgon) {
    if (!demogorgonsActivos.contains(demogorgon)) {
        demogorgonsActivos.add(demogorgon);
    }
}

public List<Demogorgon> getDemogorgonsActivos() {
    return demogorgonsActivos;
}

public List<Demogorgon> getBosqueUpsideDownDemogorgon() {
    return BosqueUpsideDownDemogorgon;
}

public List<Demogorgon> getLaboratorioUpsideDownDemogorgon() {
    return LaboratorioUpsideDownDemogorgon;
}

public List<Demogorgon> getAlcantarilladoUpsideDownDemogorgon() {
    return AlcantarilladoUpsideDownDemogorgon;
}

public List<Demogorgon> getCentroComercialUpsideDownDemogorgon() {
    return CentroComercialUpsideDownDemogorgon;
}

public List<Nino> getRadioWSQK() {
    return RadioWSQK;
}

public Semaphore getPortalAlcantarillaSemaphore() {
    return portalAlcantarillaSemaphore;
}

public Semaphore getPortalCentroComercialSemaphore() {
    return portalCentroComercialSemaphore;
}

public Semaphore getPortalLaboratorioSemaphore() {
    return portalLaboratorioSemaphore;
}

public Semaphore getPortalBosqueSemaphore() {
    return portalBosqueSemaphore;
}

public List<Nino> getLaboratorioUpsideDown() {
    return LaboratorioUpsideDown;
}

public List<Nino> getCentroComercialUpsideDown() {
    return CentroComercialUpsideDown;
}

public List<Nino> getAlcantarilladoUpsideDown() {
    return AlcantarilladoUpsideDown;
}

public List<Nino> getBosqueUpsideDown() {
    return BosqueUpsideDown;
}

public LinkedBlockingQueue<Nino> getPortalNormalCentroComercial() {
    return portalNormalCentroComercial;
}

public LinkedBlockingQueue<Nino> getPortalNormalBosque() {
    return portalNormalBosque;
}

public LinkedBlockingQueue<Nino> getPortalUpsideDownBosque() {
    return portalUpsideDownBosque;
}

public LinkedBlockingQueue<Nino> getPortalNormalLaboratorio() {
    return portalNormalLaboratorio;
}

public LinkedBlockingQueue<Nino> getPortalUpsideDownLaboratorio() {
    return portalUpsideDownLaboratorio;
}

```

```

public LinkedBlockingQueue<Nino> getPortalUpsideDownCentroComercial() {
    return portalUpsideDownCentroComercial;
}

public LinkedBlockingQueue<Nino> getPortalNormalAlcantarillado() {
    return portalNormalAlcantarillado;
}

public LinkedBlockingQueue<Nino> getPortalUpsideDownAlcantarillado() {
    return portalUpsideDownAlcantarillado;
}

public List<Nino> getCallePrincipal() {
    return callePrincipal;
}

public List<Nino> getSotanoByers() {
    return sotanoByers;
}

public CyclicBarrier getIrBosque() {
    return irBosque;
}

public CyclicBarrier getIrLaboratorio() {
    return irLaboratorio;
}

public CyclicBarrier getIrCentroComercial() {
    return irCentroComercial;
}

public CyclicBarrier getIrAlcantarillado() {
    return irAlcantarillado;
}

public List<Nino> getLaColmena() {
    return LaColmena;
}

public void takeConDelay(LinkedBlockingQueue<Nino> queue) throws InterruptedException {
    Semaphore semaphore = getSemaphoreAdecuado(queue);
    if (queue == getPortalNormalAlcantarillado() || queue == getPortalNormalLaboratorio() || queue == getPortalNormalCentroComercial() || queue
== getPortalNormalBosque()){
        if (getPortalPareja(queue).isEmpty()){
            try{
                semaphore.acquire();
                queue.poll(1, TimeUnit.SECONDS);
                semaphore.release();
            }catch(InterruptedException e){
                Thread.currentThread().interrupt();
            }
        }
    }else{
        try{
            semaphore.acquire();
            queue.poll(1, TimeUnit.SECONDS);
            semaphore.release();
        }catch(InterruptedException e){
            Thread.currentThread().interrupt();
        }
    }
}

public LinkedBlockingQueue<Nino> getPortalPareja(LinkedBlockingQueue<Nino> portal) {
    if (portal == getPortalNormalAlcantarillado()) {
        return getPortalUpsideDownAlcantarillado();
    }
    else if (portal == getPortalUpsideDownAlcantarillado()) {
        return getPortalNormalAlcantarillado();
    }
    else if (portal == getPortalUpsideDownCentroComercial()) {
        return getPortalNormalCentroComercial();
    }
    else if (portal == getPortalNormalCentroComercial()) {
        return getPortalUpsideDownCentroComercial();
    }
    else if (portal == getPortalUpsideDownBosque()) {
        return getPortalNormalBosque();
    }
    else if (portal == getPortalNormalBosque()) {
        return getPortalUpsideDownBosque();
    }
    else if (portal == getPortalUpsideDownLaboratorio()) {
        return getPortalNormalLaboratorio();
    }
    else {
        return getPortalUpsideDownLaboratorio();
    }
}

public Semaphore getSemaphoreAdecuado(LinkedBlockingQueue<Nino> queue) {
    if (queue == getPortalNormalAlcantarillado() || queue == getPortalUpsideDownAlcantarillado()) {
        return getPortalAlcantarillaSemaphore();
    }else if (queue == getPortalNormalCentroComercial() || queue == getPortalUpsideDownCentroComercial()) {
        return getPortalCentroComercialSemaphore();
    } else if (queue == getPortalNormalBosque() || queue == getPortalUpsideDownBosque()) {
        return getPortalBosqueSemaphore();
    }else{
        return getPortalLaboratorioSemaphore();
    }
}

```

```

    }
}

public List<Nino> getVictimas(List<Demogorgon> pareja){
    if (pareja == getAlcantarilladoUpsideDownDemogorgon()){
        return getAlcantarilladoUpsideDown();
    } else if (pareja == getCentroComercialUpsideDownDemogorgon()) {
        return getCentroComercialUpsideDown();
    } else if (pareja == getLaboratorioUpsideDownDemogorgon()) {
        return getLaboratorioUpsideDown();
    } else {
        return getBosqueUpsideDown();
    }
}

@Override
public void run() {
    while (!Thread.currentThread().isInterrupted()) {
        try {
            ControlPausa.getInstance().esperarSiPausado();
            if(!eventos.getApagonLaboratorio()) {
                // Bosque
                moverParDePortales(portalNormalBosque, portalUpsideDownBosque);
                // Laboratorio
                moverParDePortales(portalNormalLaboratorio, portalUpsideDownLaboratorio);
                // Centro comercial
                moverParDePortales(portalNormalCentroComercial, portalUpsideDownCentroComercial);
                // Alcantarillado
                moverParDePortales(portalNormalAlcantarillado, portalUpsideDownAlcantarillado);
            }

        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
        if (eventos.getIntervencionDeEleven()) {
            int ninosALiberar = sangreAlmacenada.get();
            int j = Math.min(ninosALiberar, getLaColmena().size());
            if (j > 0) {
                sangreAlmacenada.addAndGet(-j);
                for (int i = 0; i < j; i++) {
                    Nino n = getLaColmena().removeFirst();
                }
                log.log("Eleven ha liberado a " + j + " niños");
            }
        }
    }
}

// Prioridad: primero los que vuelven a Hawkins (UpsideDown -> Normal)
private void moverParDePortales(LinkedBlockingQueue portalNormal,
                                LinkedBlockingQueue portalUpside) throws InterruptedException {

    if (!portalUpside.isEmpty()) {
        // Niños que vuelven a Hawkins
        moverUnNino(portalUpside);
    } else if (!portalNormal.isEmpty()) {
        // Niños que van al UpsideDown
        moverUnNino(portalNormal);
    }
}

private void moverUnNino(LinkedBlockingQueue origen) throws InterruptedException {
    Semaphore sem = getSemaphoreAdecuado(origen);

    if (!sem.tryAcquire()) {
        return; // otro niño está cruzando este portal
    }

    try {
        Object o = origen.poll(1, TimeUnit.MILLISECONDS);
        if (o == null) {
            return;
        }

        // tiempo de cruce ~1 segundo
        ControlPausa.getInstance().dormir(1000);
    } finally {
        sem.release();
    }
}
}
}

```

main.java

```

import javax.swing.*;
import java.util.concurrent.Semaphore;

public class main {
    public static void main(String[] args) throws InterruptedException {
        Log log = Log.getInstance();
        log.log("***Empieza la batalla de hawkins.***");
        EventosAleatorios eventos = new EventosAleatorios(log);
        eventos.start();
        Lugares lugares = new Lugares(eventos, log);
        lugares.start(); // Pongo a funcionar los portales
        ServidorRMI.iniciar(lugares, eventos, log);
        Semaphore semaforo = new Semaphore(1);
        Demogorgon demogorgonAlpha = new Demogorgon(lugares, eventos, log, semaforo);
        demogorgonAlpha.start();
    }
}

```

```

log.log("El demogorgon Alpha D0000 ha sido creado");
CrearNinos creadorDeNinos = new CrearNinos(lugares,eventos,log);
creadorDeNinos.start();
CrearDemogorgons creadorDeDemogorgons = new CrearDemogorgons(lugares,eventos,log,semaforo);
creadorDeDemogorgons.start();
SwingUtilities.invokeLater(() -> new SimulacionGUI(log, eventos,lugares,creadorDeNinos,creadorDeDemogorgons).setVisible(true));

}

}

```

Nino.java

```

import java.util.List;
import java.util.Random;
import java.util.concurrent.*;

public class Nino extends Thread {

    private String Nombre; //NXXXX
    private Boolean haEstadoEnColmena=false;
    private int Sangre=0;
    private final Random rng = new Random();
    private Lugares lugares;
    private LinkedBlockingQueue<Nino> portalAIUD;
    private List<Nino> destino;
    private EventosAleatorios eventosAleatorios;
    private Log log;

    public Nino(String Nombre,Lugares lugares,EventosAleatorios eventosAleatorios,Log log) {
        this.Nombre = Nombre;
        this.lugares = lugares;
        this.eventosAleatorios = eventosAleatorios;
        this.log = log;
    }

    public void setSangre(int Sangre) {
        this.Sangre = Sangre;
    }

    public int getSangre() {
        return Sangre;
    }

    public boolean sufreAtaque(long duracionMs) throws InterruptedException {
        ControlPausa.getInstance().dormir(duracionMs);
        boolean aguanta = rng.nextInt(3)!=1;
        if(haEstadoEnColmena){ return false; }
        else{
            if(!aguanta) {
                haEstadoEnColmena=true;
            }
        }
        return aguanta;
        /* Devuelve true si resiste el ataque 2/3 */
    }

    public String getNombre() {
        return Nombre;
    }

    @Override
    public void run() {
        while (true){
            try{
                ControlPausa.getInstance().esperarSiPausado();
                lugares.getCallePrincipal().add(this);//Inician en la calle principal de Hawkins
                log.log("El niño "+ Nombre +" ha llegado a la calle principal de Hawkins");
                CicloNino();
            }
            catch (Exception e){
                System.out.println(e);
            }
        }
    }

    public void CicloNino() throws BrokenBarrierException, InterruptedException {
        try {
            while (!Thread.currentThread().isInterrupted()) {
                ControlPausa.getInstance().esperarSiPausado();
                lugares.getCallePrincipal().remove(this);
                lugares.getSotanoByers().add(this);

                int CooldownPreparacion = 1000 + (int) (Math.random() * (2000 - 1000 + 1)); //Creación del Cooldown
                log.log("El niño "+ Nombre +" ha llegado al sotano byers, recibirá ordenes allí durante "+ CooldownPreparacion/1000+" segundos.");
                ControlPausa.getInstance().dormir(CooldownPreparacion); //Espera entre 1 y 2 segundos
                //Elección aleatoria de Portal
                int Portal = (int) (Math.random() * (4)); //Selección Portal aleatorio
                switch (Portal) { //Espera a grupo deseado
                    case 0:
                        log.log("El niño "+Nombre+" irá hacia al bosque, esperando su equipo.");
                        lugares.getIrBosque().await();
                        portalAIUD = lugares.getPortalNormalBosque();
                        destino = lugares.getBosqueUpsideDown();
                        break;
                    case 1:
                        log.log("El niño "+Nombre+" irá hacia al centro comercial, esperando su equipo.");
                        lugares.getIrCentroComercial().await();
                        portalAIUD = lugares.getPortalNormalCentroComercial();
                        destino= lugares.getCentroComercialUpsideDown();
                        break;
                    case 2:
                        log.log("El niño "+Nombre+" irá hacia al laboratorio, esperando su equipo.");

```

```

        lugares.getIrLaboratorio().await();
        portalAIUD = lugares.getPortalNormalLaboratorio();
        destino = lugares.getLaboratorioUpsideDown();
        break;
    case 3:
        log.log("El niño "+Nombre+" irá hacia al alcantarillado, esperando su equipo.");
        lugares.getIrAlcantarillado().await();
        portalAIUD = lugares.getPortalNormalAlcantarillado();
        destino = lugares.getAlcantarilladoUpsideDown();
        break;
    }
    lugares.getSotanoByers().remove(this);
    portalAIUD.add(this);
    log.log("El niño "+Nombre+" ha llegado a su portal, esperando a que haya hueco para ir al upside down.");
    while(portalAIUD.contains(this)){
        ControlPausa.getInstance().dormir(1);
    }

    destino.add(this);
    int TiempoEnUpsideDown = 3000 + (int) (Math.random() * (5000 - 3000 + 1));
    if(eventosAleatorios.getTormentaUpsideDown()){
        TiempoEnUpsideDown = TiempoEnUpsideDown*2;
    }
    log.log("El niño "+Nombre+" ha pasado al upside down permanecerá allí "+ TiempoEnUpsideDown/1000 +" segundos.");
    ControlPausa.getInstance().dormir(TiempoEnUpsideDown);
    if(haEstadoEnColmena){
        log.log("El niño "+ Nombre +" ha sido atrapado por un demogorgon, actualmente en la colmena");
        while(lugares.getLaColmena().contains(this)){
            ControlPausa.getInstance().dormir(1);
        }
        log.log("El niño "+ Nombre +" ha sido liberado de la colmena");
    }else {
        LinkedBlockingQueue<Nino> portalAIMN = lugares.getPortalPareja(portalAIUD);
        destino.remove(this);
        portalAIMN.add(this);
        log.log("El niño "+Nombre+" está en el portal del upside down, esperando a poder pasar");
        while (portalAIMN.contains(this)) {
            ControlPausa.getInstance().dormir(1);
        }
    }

    log.log("El niño "+Nombre+" ha vuelto del upside down, reportará a radio WSQK");
    lugares.getRadioWSQK().add(this);
    if (!haEstadoEnColmena){
        setSangre(1);
        log.log("El niño "+Nombre+" deposita uno de sangre en el bote, llevan por ahora "+ lugares.sangreAlmacenada.addAndGet(1));
    }else{
        setSangre(0);
        log.log("El niño "+Nombre+" ha vuelto sin sangre");
    }
    haEstadoEnColmena=false;
    int TiempoWSQK = 2000 + (int) (Math.random() * (4000 - 2000 + 1));
    ControlPausa.getInstance().dormir(TiempoWSQK);
    lugares.getRadioWSQK().remove(this);
    lugares.getCallePrincipal().add(this);

    int TiempoCallePrincipal = 3000 + (int) (Math.random() * (5000 - 3000 + 1));
    log.log("El niño "+Nombre+" irá a la calle principal durante "+TiempoCallePrincipal/1000 +" segundos, para evitar sospechas del
ejercicio");
    ControlPausa.getInstance().dormir(TiempoCallePrincipal);
}
} catch (Exception e) {
    throw new RuntimeException(e);
}
}
}

```

ServidorRMI.java

```

import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;

public class ServidorRMI {
    public static final String NOMBRE_SERVICIO = "HawkinsService";
    public static final int PUERTO = 1099;

    private ServidorRMI() {}

    public static void iniciar(Lugares lugares, EventosAleatorios eventos, Log log) {
        try {
            try {
                LocateRegistry.createRegistry(PUERTO);
                log.log("Registro RMI iniciado en el puerto " + PUERTO);
            } catch (Exception e) {
                log.log("El registro RMI ya estaba iniciado o no se pudo crear: " + e.getMessage());
            }

            HawkinsService servicio = new HawkinsServiceImpl(lugares, eventos, log);
            Naming.rebind("rmi://localhost:" + PUERTO + "/" + NOMBRE_SERVICIO, servicio);
            log.log("Servidor RMI disponible en rmi://localhost:" + PUERTO + "/" + NOMBRE_SERVICIO);
        } catch (Exception e) {
            log.log("ERROR iniciando servidor RMI: " + e.getMessage());
            e.printStackTrace();
        }
    }
}

```

SimulacionGUI.java

```
import javax.swing.*;
import javax.swing.border.*;
import java.awt.*;
import javax.swing.*;
import javax.swing.border.*;
import java.awt.*;
import java.awt.event.*;

/**
 * SimulacionGUI.java — Interfaz Swing para "La Batalla de Hawkins"
 *
 * Diseño simple y claro: fondo blanco, paneles bien delimitados,
 * actualización automática cada 500 ms.
 *
 * Requiere que Log.java tenga setGuiCallback() (ya incluido en el Log entregado).
 */
public class SimulacionGUI extends JFrame {

    // — Dependencias de simulación —————
    private final Log log;
    private final EventosAleatorios eventos;
    private final Lugares lugares;
    private final CrearNinos creadorNinos;
    private final CrearDemogorgons creadorDemogorgons;

    // — Colores sencillos —————
    private static final Color C_BG = new Color(245, 245, 248);
    private static final Color C_HAWKINS = new Color(220, 235, 255);
    private static final Color C_UPSIDE = new Color(255, 220, 220);
    private static final Color C_PORTALES = new Color(220, 255, 230);
    private static final Color C_EVENTO = new Color(255, 243, 180);
    private static final Color C_COLMENA = new Color(255, 200, 200);
    private static final Color C_HEADER = new Color(50, 50, 80);
    private static final Color C_EVENT_ON = new Color(220, 60, 60);
    private static final Color C_EVENT_OFF = new Color(160, 160, 160);
    private static final Font F_TITLE = new Font("SansSerif", Font.BOLD, 13);
    private static final Font F_NORMAL = new Font("SansSerif", Font.PLAIN, 12);
    private static final Font F_BIG = new Font("SansSerif", Font.BOLD, 22);
    private static final Font F_MONO = new Font("Monospaced", Font.PLAIN, 11);

    // — Labels dinámicos —————
    // Resumen global
    private JLabel lblNinosCreados, lblDemogorgonsActivos, lblSangre, lblVictimas;

    // Hawkins
    private JLabel lblCalle, lblSotano, lblRadio;

    // Portales (Normal→UD | UD→Normal)
    private JLabel lblPBN, lblPBUD; // Bosque
    private JLabel lblPLN, lblPLUD; // Laboratorio
    private JLabel lblPCN, lblPCUD; // Centro Comercial
    private JLabel lblPAN, lblPAUD; // Alcantarillado

    // Upside Down
    private JLabel lblBosqueNinos, lblLabNinos, lblCentroNinos, lblAlcantNinos;
    private JLabel lblBosqueDem, lblLabDem, lblCentroDem, lblAlcantDem;
    private JLabel lblColmena;

    // Eventos
    private JLabel lblEvTormenta, lblEvRed, lblEvEleven, lblEvApagon;
    private JLabel lblEventoActivo;

    // Log
    private JTextArea areaLog;

    // Timer de refresco
    private Timer timerRefresh;

    // —————
    public SimulacionGUI(Log log, EventosAleatorios eventos, Lugares lugares,
        CrearNinos creadorNinos, CrearDemogorgons creadorDemogorgons) {
        super("La Batalla de Hawkins — Simulación Concurrente");
        this.log = log;
        this.eventos = eventos;
        this.lugares = lugares;
        this.creadorNinos = creadorNinos;
        this.creadorDemogorgons = creadorDemogorgons;

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(1100, 760);
        setMinimumSize(new Dimension(900, 600));
        setLocationRelativeTo(null);
        getContentPane().setBackground(C_BG);
        setLayout(new BorderLayout(6, 6));

        add(buildHeader(), BorderLayout.NORTH);
        add(buildMain(), BorderLayout.CENTER);
        add(buildLogPanel(), BorderLayout.SOUTH);

        // Conectar log a la GUI
        Log.setGuiCallback(msg -> SwingUtilities.invokeLater(() -> appendLog(msg)));

        // Refresco automático
        timerRefresh = new Timer(500, e -> refreshStats());
        timerRefresh.start();
    }

    // =====
    // CABECERA — resumen global
    // =====
    private JPanel buildHeader() {
```

```

JPanel p = new JPanel(new GridLayout(1, 4, 8, 0));
p.setBackground(C_HEADER);
p.setBorder(new EmptyBorder(10, 14, 10, 14));

lblNinosCreados = bigStat("Niños creados", "0", new Color(100, 180, 255));
lblDemogorgonsActivos = bigStat("Demogorgons", "0", new Color(255, 120, 120));
lblSangre = bigStat("Sangre Vecna", "0", new Color(255, 220, 80));
lblVictimas = bigStat("En Colmena", "0", new Color(220, 100, 255));

p.add(wrapStat("NIÑOS CREADOS", lblNinosCreados, new Color(100, 180, 255)));
p.add(wrapStat("DEMOGORGONS", lblDemogorgonsActivos, new Color(255, 120, 120)));
p.add(wrapStat("SANGRE DE VECNA", lblSangre, new Color(255, 220, 80)));
p.add(wrapStat("EN COLMENA", lblVictimas, new Color(220, 100, 255)));
return p;
}

private JLabel bigStat(String name, String val, Color c) {
    JLabel l = new JLabel(val, SwingConstants.CENTER);
    l.setFont(F_BIG);
    l.setForeground(c);
    return l;
}

private JPanel wrapStat(String titulo, JLabel valor, Color c) {
    JPanel p = new JPanel(new BorderLayout(2, 2));
    p.setOpaque(false);
    JLabel t = new JLabel(titulo, SwingConstants.CENTER);
    t.setFont(new Font("SansSerif", Font.BOLD, 10));
    t.setForeground(new Color(180, 180, 200));
    p.add(t, BorderLayout.NORTH);
    p.add(valor, BorderLayout.CENTER);
    return p;
}

// =====
// PANEL PRINCIPAL — 3 columnas
// =====
private JPanel buildMain() {
    JPanel p = new JPanel(new GridLayout(1, 3, 8, 0));
    p.setBackground(C_BG);
    p.setBorder(new EmptyBorder(6, 10, 6, 10));
    p.add(buildHawkinsCol());
    p.add(buildPortalesCol());
    p.add(buildUpsideDownCol());
    return p;
}

// — Columna izquierda: Hawkins + Eventos —————
private JPanel buildHawkinsCol() {
    JPanel col = new JPanel();
    col.setLayout(new BoxLayout(col, BoxLayout.Y_AXIS));
    col.setBackground(C_BG);

    // Hawkins
    JPanel pHawkins = section("HAWKINS — MUNDO NORMAL", C_HAWKINS);
    lblCalle = rowLabel("Calle Principal:");
    lblSotano = rowLabel("Sótano Byers:");
    lblRadio = rowLabel("Radio WSQK:");
    addRows(pHawkins,
        labelRow("Calle Principal:", lblCalle),
        labelRow("Sótano Byers:", lblSotano),
        labelRow("Radio WSQK:", lblRadio));
    col.add(pHawkins);
    col.add(Box.createVerticalStrut(8));

    // Eventos
    JPanel pEv = section("EVENTO GLOBAL ACTIVO", C_EVENTO);
    lblEventoActivo = new JLabel("Sin evento activo", SwingConstants.CENTER);
    lblEventoActivo.setFont(F_TITLE);
    lblEventoActivo.setForeground(C_EVENT_OFF);
    lblEventoActivo.setAlignmentX(Component.CENTER_ALIGNMENT);

    JPanel grid = new JPanel(new GridLayout(2, 2, 4, 4));
    grid.setOpaque(false);
    lblEvTormenta = evLabel("☐ Tormenta UD");
    lblEvRed = evLabel("☐ Red Mental");
    lblEvEleven = evLabel("✦ Eleven");
    lblEvApagon = evLabel("☐ Apagón Lab.");
    grid.add(lblEvTormenta);
    grid.add(lblEvRed);
    grid.add(lblEvEleven);
    grid.add(lblEvApagon);

    pEv.add(lblEventoActivo);
    pEv.add(Box.createVerticalStrut(6));
    pEv.add(grid);
    col.add(pEv);
    col.add(Box.createVerticalGlue());
    return col;
}

// — Columna central: Portales —————
private JPanel buildPortalesCol() {
    JPanel col = new JPanel();
    col.setLayout(new BoxLayout(col, BoxLayout.Y_AXIS));
    col.setBackground(C_BG);

    JPanel pPort = section("PORTALES (un niño a la vez)", C_PORTALES);

    // Encabezado de tabla
    JPanel hdr = new JPanel(new GridLayout(1, 3, 4, 0));
    hdr.setOpaque(false);
    hdr.add(headerCell("ZONA"));

```



```

hdr.add(headerCell("→ UD (Normal)"));
hdr.add(headerCell("← HW (UD)"));
hdr.setMaximumSize(new Dimension(Integer.MAX_VALUE, 22));
pPort.add(hdr);
pPort.add(Box.createVerticalStrut(4));

lbpbn = valLabel("0"); lbpbud = valLabel("0");
lbpbn = valLabel("0"); lbpbud = valLabel("0");
lbpbn = valLabel("0"); lbpbud = valLabel("0");
lbpbn = valLabel("0"); lbpbud = valLabel("0");
lbpbn = valLabel("0"); lbpbud = valLabel("0");

pPort.add(portalRow("Bosque", lbpbn, lbpbud));
pPort.add(Box.createVerticalStrut(3));
pPort.add(portalRow("Laboratorio", lbpbn, lbpbud));
pPort.add(Box.createVerticalStrut(3));
pPort.add(portalRow("Centro Comercial", lbpbn, lbpbud));
pPort.add(Box.createVerticalStrut(3));
pPort.add(portalRow("Alcantarillado", lbpbn, lbpbud));

col.add(pPort);
col.add(Box.createVerticalGlue());
return col;
}

// — Columna derecha: Upside Down —————
private JPanel buildUpsideDownCol() {
    JPanel col = new JPanel();
    col.setLayout(new BoxLayout(col, BoxLayout.Y_AXIS));
    col.setBackground(C_BG);

    JPanel pUD = section("UPSIDE DOWN", C_UPSIDE);

    // Encabezado de tabla
    JPanel hdr = new JPanel(new GridLayout(1, 3, 4, 0));
    hdr.setOpaque(false);
    hdr.add(headerCell("ZONA"));
    hdr.add(headerCell("Niños □"));
    hdr.add(headerCell("Demogorgons □"));
    hdr.setMaximumSize(new Dimension(Integer.MAX_VALUE, 22));
    pUD.add(hdr);
    pUD.add(Box.createVerticalStrut(4));

    lblBosqueNinos = valLabel("0"); lblBosqueDem = valLabel("0");
    lblLabNinos = valLabel("0"); lblLabDem = valLabel("0");
    lblCentroNinos = valLabel("0"); lblCentroDem = valLabel("0");
    lblAlcantNinos = valLabel("0"); lblAlcantDem = valLabel("0");

    pUD.add(zonaRow("Bosque", lblBosqueNinos, lblBosqueDem));
    pUD.add(Box.createVerticalStrut(3));
    pUD.add(zonaRow("Laboratorio", lblLabNinos, lblLabDem));
    pUD.add(Box.createVerticalStrut(3));
    pUD.add(zonaRow("Centro Comercial", lblCentroNinos, lblCentroDem));
    pUD.add(Box.createVerticalStrut(3));
    pUD.add(zonaRow("Alcantarillado", lblAlcantNinos, lblAlcantDem));

    // Colmena
    pUD.add(Box.createVerticalStrut(10));
    JSeparator sep = new JSeparator();
    sep.setMaximumSize(new Dimension(Integer.MAX_VALUE, 2));
    pUD.add(sep);
    pUD.add(Box.createVerticalStrut(6));

    lblColmena = new JLabel("COLMENA: 0 niños capturados", SwingConstants.CENTER);
    lblColmena.setFont(F_TITLE);
    lblColmena.setForeground(new Color(180, 40, 40));
    lblColmena.setAlignmentX(Component.CENTER_ALIGNMENT);
    pUD.add(lblColmena);

    col.add(pUD);
    col.add(Box.createVerticalGlue());
    return col;
}

// =====
// PANEL DE LOG
// =====
private JPanel buildLogPanel() {
    JPanel p = new JPanel(new BorderLayout(4, 4));
    p.setBackground(C_BG);
    p.setBorder(new EmptyBorder(0, 10, 8, 10));

    JLabel titulo = new JLabel("REGISTRO DE EVENTOS (hawkins.txt)");
    titulo.setFont(F_TITLE);
    titulo.setForeground(C_HEADER);

    areaLog = new JTextArea(7, 80);
    areaLog.setFont(F_MONO);
    areaLog.setEditable(false);
    areaLog.setBackground(new Color(30, 30, 40));
    areaLog.setForeground(new Color(180, 255, 180));
    areaLog.setCaretColor(Color.WHITE);
    JScrollPane scroll = new JScrollPane(areaLog);
    scroll.setBorder(BorderFactory.createLineBorder(new Color(80, 80, 100), 1));

    p.add(titulo, BorderLayout.NORTH);
    p.add(scroll, BorderLayout.CENTER);
    return p;
}

// =====
// REFRESCO DE DATOS
// =====
private void refreshStats() {

```

```

// Resumen global
int ninos = creadorNinos.contadorNinos.get();
int demos = creadorDemogorgons.contadorDemogorgons.get();
lblNinosCreados.setText(String.valueOf(ninos));
lblDemogorgonsActivos.setText(String.valueOf(demos));
lblSangre.setText(String.valueOf(lugares.sangreAlmacenada.get()));
int colmena = lugares.getLaColmena().size();
lblVictimas.setText(String.valueOf(colmena));

// Hawkins
lblCalle.setText(String.valueOf(lugares.getCallePrincipal().size()));
lblSotano.setText(String.valueOf(lugares.getSotanoByers().size()));
lblRadio.setText(String.valueOf(lugares.getRadioWSQK().size()));

// Portales
lblPBN.setText(String.valueOf(lugares.getPortalNormalBosque().size()));
lblPBUD.setText(String.valueOf(lugares.getPortalUpsideDownBosque().size()));
lblPLN.setText(String.valueOf(lugares.getPortalNormalLaboratorio().size()));
lblPLUD.setText(String.valueOf(lugares.getPortalUpsideDownLaboratorio().size()));
lblPCN.setText(String.valueOf(lugares.getPortalNormalCentroComercial().size()));
lblPCUD.setText(String.valueOf(lugares.getPortalUpsideDownCentroComercial().size()));
lblPAN.setText(String.valueOf(lugares.getPortalNormalAlcantarillado().size()));
lblPAUD.setText(String.valueOf(lugares.getPortalUpsideDownAlcantarillado().size()));

// Upside Down
lblBosqueNinos.setText(String.valueOf(lugares.getBosqueUpsideDown().size()));
lblLabNinos.setText(String.valueOf(lugares.getLaboratorioUpsideDown().size()));
lblCentroNinos.setText(String.valueOf(lugares.getCentroComercialUpsideDown().size()));
lblAlcantNinos.setText(String.valueOf(lugares.getAlcantarilladoUpsideDown().size()));
lblBosqueDem.setText(String.valueOf(lugares.getBosqueUpsideDownDemogorgon().size()));
lblLabDem.setText(String.valueOf(lugares.getLaboratorioUpsideDownDemogorgon().size()));
lblCentroDem.setText(String.valueOf(lugares.getCentroComercialUpsideDownDemogorgon().size()));
lblAlcantDem.setText(String.valueOf(lugares.getAlcantarilladoUpsideDownDemogorgon().size()));

// Colmena
lblColmena.setText("COLMENA: " + colmena + " niño(s) capturado(s)");
lblColmena.setForeground(colmena > 0 ? new Color(180, 40, 40) : new Color(100, 140, 100));

// Eventos
boolean tormenta = eventos.getTormentaUpsideDown();
boolean red = eventos.getLaRedMental();
boolean eleven = eventos.getIntervencionDeEleven();
boolean apagon = eventos.getApagonLaboratorio();

setEvento(lblEvTormenta, tormenta);
setEvento(lblEvRed, red);
setEvento(lblEvEleven, eleven);
setEvento(lblEvApagon, apagon);

if (tormenta) { lblEventoActivo.setText("☐ TORMENTA DEL UPSIDE DOWN"); lblEventoActivo.setForeground(new Color(200, 120, 0)); }
else if (red) { lblEventoActivo.setText("☐ LA RED MENTAL"); lblEventoActivo.setForeground(new Color(100, 0, 180)); }
else if (eleven) { lblEventoActivo.setText("✱ INTERVENCIÓN DE ELEVEN"); lblEventoActivo.setForeground(new Color(0, 140, 200)); }
else if (apagon) { lblEventoActivo.setText("☐ APAGÓN DEL LABORATORIO"); lblEventoActivo.setForeground(new Color(180, 40, 40)); }
else { lblEventoActivo.setText("Sin evento activo"); lblEventoActivo.setForeground(C_EVENT_OFF); }
}

private void setEvento(JLabel lbl, boolean activo) {
    lbl.setForeground(activo ? C_EVENT_ON : C_EVENT_OFF);
    lbl.setFont(activo
        ? new Font("SansSerif", Font.BOLD, 11)
        : new Font("SansSerif", Font.PLAIN, 11));
}

private void appendLog(String msg) {
    areaLog.append(msg + "\n");
    // Limitar a 500 líneas
    if (areaLog.getLineCount() > 520) {
        try {
            int end = areaLog.getLineEndOffset(areaLog.getLineCount() - 501);
            areaLog.getDocument().remove(0, end);
        } catch (Exception ignored) {}
    }
    areaLog.setCaretPosition(areaLog.getDocument().getLength());
}

// =====
// CONSTRUCTORES DE COMPONENTES AUXILIARES
// =====

/** Panel con título y borde coloreado, BoxLayout vertical */
private JPanel section(String titulo, Color bg) {
    JPanel p = new JPanel();
    p.setLayout(new BoxLayout(p, BoxLayout.Y_AXIS));
    p.setBackground(bg);
    p.setBorder(BorderFactory.createCompoundBorder(
        BorderFactory.createLineBorder(bg.darker(), 1),
        new EmptyBorder(8, 10, 8, 10)
    ));
    JLabel t = new JLabel(titulo);
    t.setFont(F_TITLE);
    t.setForeground(C_HEADER);
    t.setAlignmentX(Component.LEFT_ALIGNMENT);
    p.add(t);
    p.add(Box.createVerticalStrut(8));
    return p;
}

/** Fila "Etiqueta: [valor]" */
private JPanel labelRow(String etiqueta, JLabel valor) {
    JPanel row = new JPanel(new BorderLayout(6, 0));
    row.setOpaque(false);
    row.setMaximumSize(new Dimension(Integer.MAX_VALUE, 22));
    JLabel lbl = new JLabel(etiqueta);

```

```

        lbl.setFont(F_NORMAL);
        lbl.setForeground(C_HEADER);
        row.add(lbl, BorderLayout.WEST);
        row.add(valor, BorderLayout.EAST);
        return row;
    }

    /** Fila de 3 celdas para la tabla de portales */
    private JPanel portalRow(String zona, JLabel normal, JLabel upside) {
        JPanel row = new JPanel(new GridLayout(1, 3, 4, 0));
        row.setOpaque(false);
        row.setMaximumSize(new Dimension(Integer.MAX_VALUE, 24));
        JLabel z = new JLabel(zona);
        z.setFont(F_NORMAL);
        z.setForeground(C_HEADER);
        row.add(z);
        row.add(normal);
        row.add(upside);
        return row;
    }

    /** Fila de 3 celdas para la tabla del Upside Down */
    private JPanel zonaRow(String zona, JLabel ninos, JLabel demos) {
        JPanel row = new JPanel(new GridLayout(1, 3, 4, 0));
        row.setOpaque(false);
        row.setMaximumSize(new Dimension(Integer.MAX_VALUE, 24));
        JLabel z = new JLabel(zona);
        z.setFont(F_NORMAL);
        z.setForeground(C_HEADER);
        row.add(z);
        row.add(ninos);
        row.add(demos);
        return row;
    }

    /** Celda de cabecera de tabla */
    private JLabel headerCell(String txt) {
        JLabel l = new JLabel(txt, SwingConstants.CENTER);
        l.setFont(new Font("SansSerif", Font.BOLD, 11));
        l.setForeground(new Color(80, 80, 120));
        return l;
    }

    /** Label de valor numérico centrado */
    private JLabel valLabel(String txt) {
        JLabel l = new JLabel(txt, SwingConstants.CENTER);
        l.setFont(new Font("SansSerif", Font.BOLD, 13));
        l.setForeground(new Color(40, 80, 160));
        return l;
    }

    /** Label de zona Hawkins (valor alineado a la derecha) */
    private JLabel rowLabel(String txt) {
        JLabel l = new JLabel(txt, SwingConstants.RIGHT);
        l.setFont(new Font("SansSerif", Font.BOLD, 13));
        l.setForeground(new Color(40, 80, 160));
        return l;
    }

    /** Label de evento (apagado por defecto) */
    private JLabel evLabel(String txt) {
        JLabel l = new JLabel(txt, SwingConstants.CENTER);
        l.setFont(new Font("SansSerif", Font.PLAIN, 11));
        l.setForeground(C_EVENT_OFF);
        l.setOpaque(true);
        l.setBackground(new Color(230, 230, 235));
        l.setBorder(new EmptyBorder(4, 2, 4, 2));
        return l;
    }

    private void addRows(JPanel panel, JComponent... rows) {
        for (JComponent r : rows) {
            r.setAlignmentX(Component.LEFT_ALIGNMENT);
            panel.add(r);
            panel.add(Box.createVerticalStrut(4));
        }
    }
}

```